BMIT3173 Integrative Programming

ASSIGNMENT 202605

Student Name	: Liew Yi Ler
Student ID	: 25WMR09747
Programme	: Bachelor in Information Technology (Honours) (Information Security)
Tutorial Group	: 4
System Title	: NutriShare: Surplus Food Redistribution Platform
Chosen SDG	: SDG 2: Zero Hunger
Modules Management	: Module 1 — Surplus Food Donation Publishing & Notification

AI Tools Usage Policy & Disclosure

This assignment is governed by the **TAR UMT Policy for the Use of Artificial Intelligence (AI) (PO/111:26)** and is classified under the **YELLOW (Limited AI)** category. AI tools may be used only for the purposes listed below, not to produce the technical work, analysis, or original content being assessed.

Mandatory submission: Every student must complete and submit the **AI Usage Disclosure Form** (below) with the report, whether or not AI tools were used, by 6th September 2026. A report submitted without the Form is incomplete and may not be marked.

Permitted (Yellow) uses of AI tools:

- Language refinement (grammar, spelling, clarity) of text you wrote, without altering its technical meaning.
- Clarifying taught concepts (e.g. how a design pattern, web-service protocol, or attack works) to aid your understanding.
- Debugging help on code you wrote, where AI only locates or explains an error you then fix yourself.
- Brainstorming early ideas (e.g. SDG framings or topic angles), provided the chosen direction and its justification are your own.

Prohibited uses of AI tools:

- Generating the PHP code, design-pattern implementation, class diagrams, or web-service code being assessed.
- Producing core arguments or analysis, including the design-pattern justification, threat analysis, and secure-coding rationale.
- Uploading confidential, proprietary, or others' personal data to public AI tools (Sections 2.5 and 5 of the AI Policy).

You remain responsible for the accuracy and originality of your work; AI errors do not excuse incorrect content. The lecturer may ask you to explain or demonstrate any part to verify authorship. Undisclosed or prohibited AI use is a breach of academic integrity under Section 4 of the AI Policy. **Complete the Form below; if AI was used, list each use and cite the tool(s) in your References (APA 7th).**

AI Usage Disclosure Form

Declaration (tick one):

☐ No AI tools were used in the preparation of this report.

☑ AI tools were used as declared in the table below.

AI Tool Used (name & version)	Purpose / How It Was Used	Report Section(s) Affected
Grammarly (2026 Edition)	Used solely for language refinement, spelling verification, grammatical sentence flow, and punctuation polishing of original text written by the author without altering technical logic or analysis (Grammarly Inc., 2026).	Sections 1, 2, 4.1, 4.3, 5.1

If no AI tools were used, tick "No AI tools used". Non-disclosure breaches the AI Policy.

I declare this Form is true and complete and that my AI use complied with the AI Policy and the Yellow conditions above.

Sign: *Liew* Date: 06/09/2026

Table of Contents

1. Introduction to the System

1.1 System Overview

NutriShare is an enterprise-grade, web-based surplus food redistribution and supply chain governance platform engineered to bridge the operational gap between commercial food donors (supermarkets, hypermarkets, artisanal bakeries, hotel banquet kitchens, and restaurants) and verified Non-Governmental Organisations (NGOs), charitable foundations, and community welfare shelters.

In conventional urban food supply chains, substantial quantities of wholesome, edible food are discarded daily due to logistics coordination delays, lack of real-time inventory visibility, and manual, paper-based communication bottlenecks (Food and Agriculture Organisation [FAO], 2023). NutriShare digitalises the complete surplus food recovery lifecycle, encompassing real-time donation publishing, geolocation tagging, event-driven notifications, state-driven logistics claim lifecycles, temperature-controlled inventory management, digital collection receipts, and cryptographic activity logging.

1.2 Chosen Sustainable Development Goal (SDG)

The core socio-technical objective of NutriShare directly addresses **United Nations Sustainable Development Goal 2: Zero Hunger (UN SDG 2)**, in conjunction with **SDG 12: Responsible Consumption and Production (Target 12.3)**, established in the 2030 Agenda for Sustainable Development (United Nations, 2015).

Key SDG 2 Targets Addressed:

1. **Target 2.1:** By 2030, end hunger and ensure access by all people, in particular the poor and people in vulnerable situations, including infants, to safe, nutritious, and sufficient food all year round (United Nations, 2015).

2. **Target 2.2:** End all forms of malnutrition by facilitating the rapid redistribution of wholesome fresh produce, dairy products, and prepared meals before nutritional degradation occurs (FAO, 2023; United Nations, 2015).

3. **Target 12.3:** Halve per capita global food waste at the retail and consumer levels and reduce food losses along production and supply chains, including post-harvest losses (United Nations, 2015).

1.3 System Contribution to SDG 2 & Scope

NutriShare converts urban food surplus into direct humanitarian relief through the following concrete mechanisms:

- **Target Beneficiaries:** Underprivileged urban populations, B40 low-income households, welfare shelters, orphanages, and soup kitchens supported by verified humanitarian partners (such as Kechara Soup Kitchen, Food Rescue Foundation, and MyKasih Foundation), adhering to the equitable food distribution frameworks established by the United Nations (2015).

- **Operational Efficiency & Speed:** By converting commercial excess into instant digital listings with photo proofs, allergen tags, and precise expiry countdowns, donors can hand over surplus food to nearby NGOs within hours of shelf clearance, preventing perishable items from entering municipal landfills and mitigating methane emissions (FAO, 2023).

- **Socio-Environmental Impact Telemetry:** The platform calculates real-time quantitative metrics displaying total food rescued (in kilograms), estimated beneficiaries fed, and greenhouse gas (CO_2e) emissions prevented, transforming food rescue operations into transparent, verifiable social impact metrics (United Nations, 2015).

2. Module Description

2.1 Scope of Module 1: Donation Management Module

As the lead software engineer for **Module 1 (Donation Management Module)**, I designed and implemented the end-to-end surplus food publishing pipeline, event-driven observer notifications, multi-criteria catalogue search and filtering, interactive geolocation mapping, and automated CSV audit exportation.

2.2 Functional Breakdown & Class Paths

F1.1: Surplus Food Publishing & Multi-Media Upload

- **Description:** Enables authenticated commercial food donors to publish surplus food listings with high-resolution photos (up to 5 photos or direct image URLs), quantities, units, geolocation coordinates, and expiry dates.

- **Class Paths:**
  - **Controller:** app/Http/Controllers/DonationController.php (create, store)
  - **Form Request:** app/Http/Requests/StoreDonationRequest.php
  - **Blade View:** resources/views/donations/create.blade.php

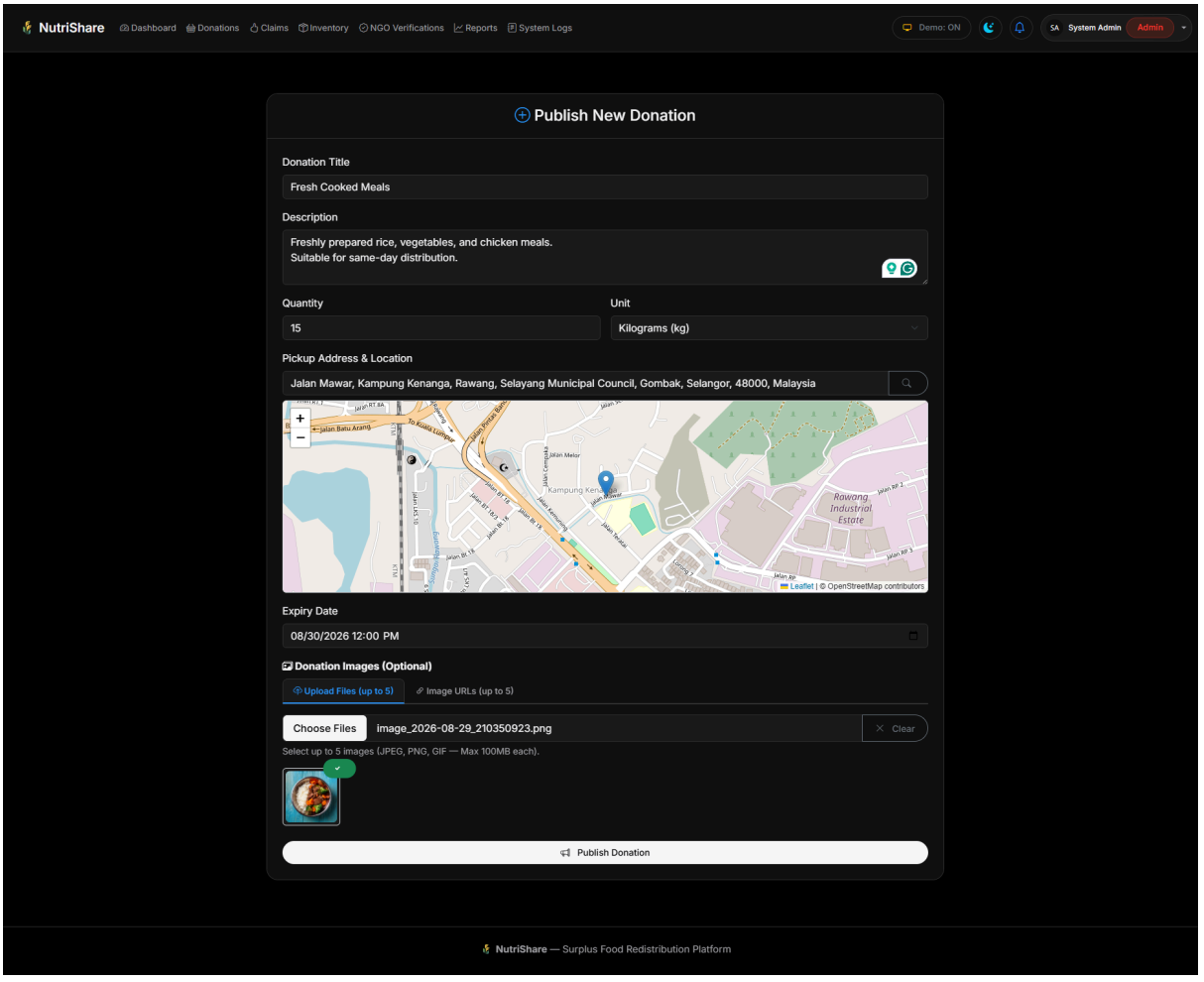


Figure 2.1: Surplus Food Publishing and Media Upload Interface

F1.2: Donation Catalogue & Parameterised Search Filter

- **Description:** Provides real-time catalogue browsing with multi-criteria searching (keyword, food category, availability status, expiry date) backed by the Repository Pattern to decouple data querying from controller presentation logic and prevent SQL injection vulnerabilities (Fowler, 2002).

- **Class Paths:**
  - **Controller:** app/Http/Controllers/DonationController.php (index)
  - **Repository:** app/Repositories/DonationRepository.php
  - **Blade View:** resources/views/donations/index.blade.php

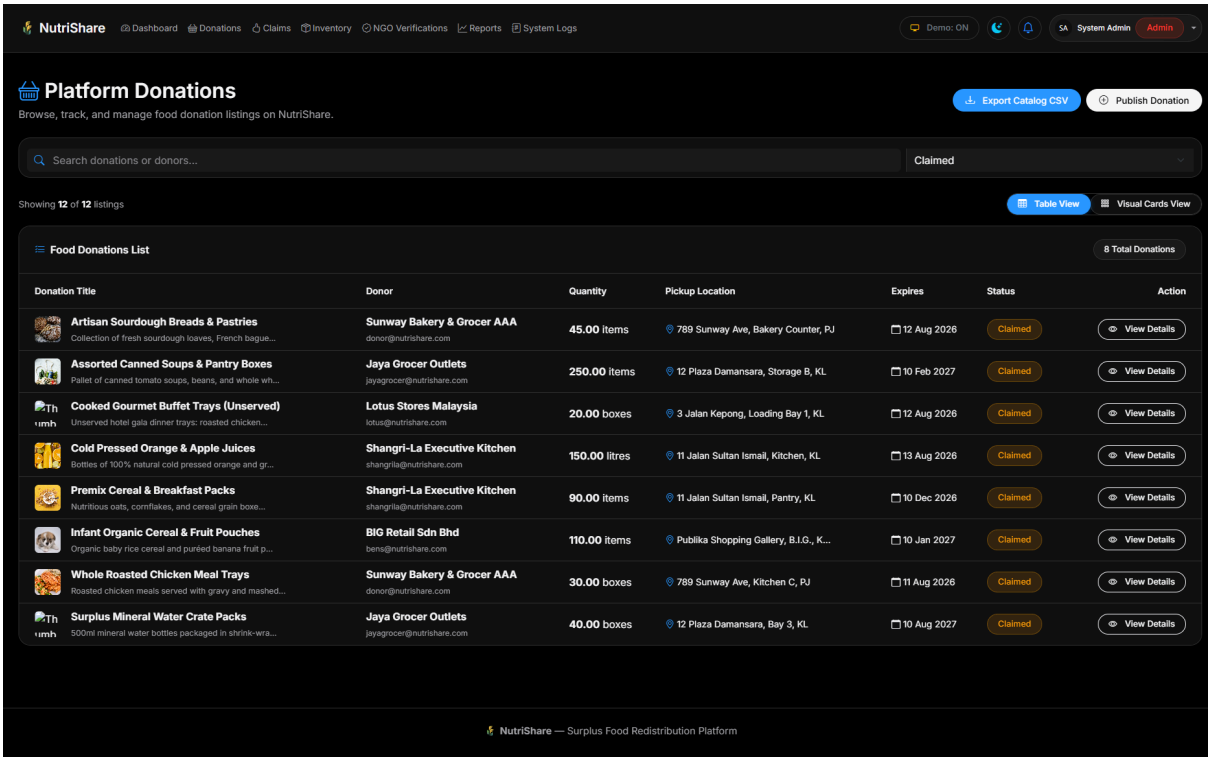


Figure 2.2: Filterable Donation Catalogue View with Visual Cards and Table View

F1.3: Interactive Geolocation Mapping & Navigation

- **Description:** Renders an interactive Leaflet.js OpenStreetMap visualizer pinpointing exact store/warehouse pickup coordinates to assist NGO logistics dispatchers in planning vehicle routes.

- **Class Paths:**
  - **Blade View:** resources/views/donations/show.blade.php

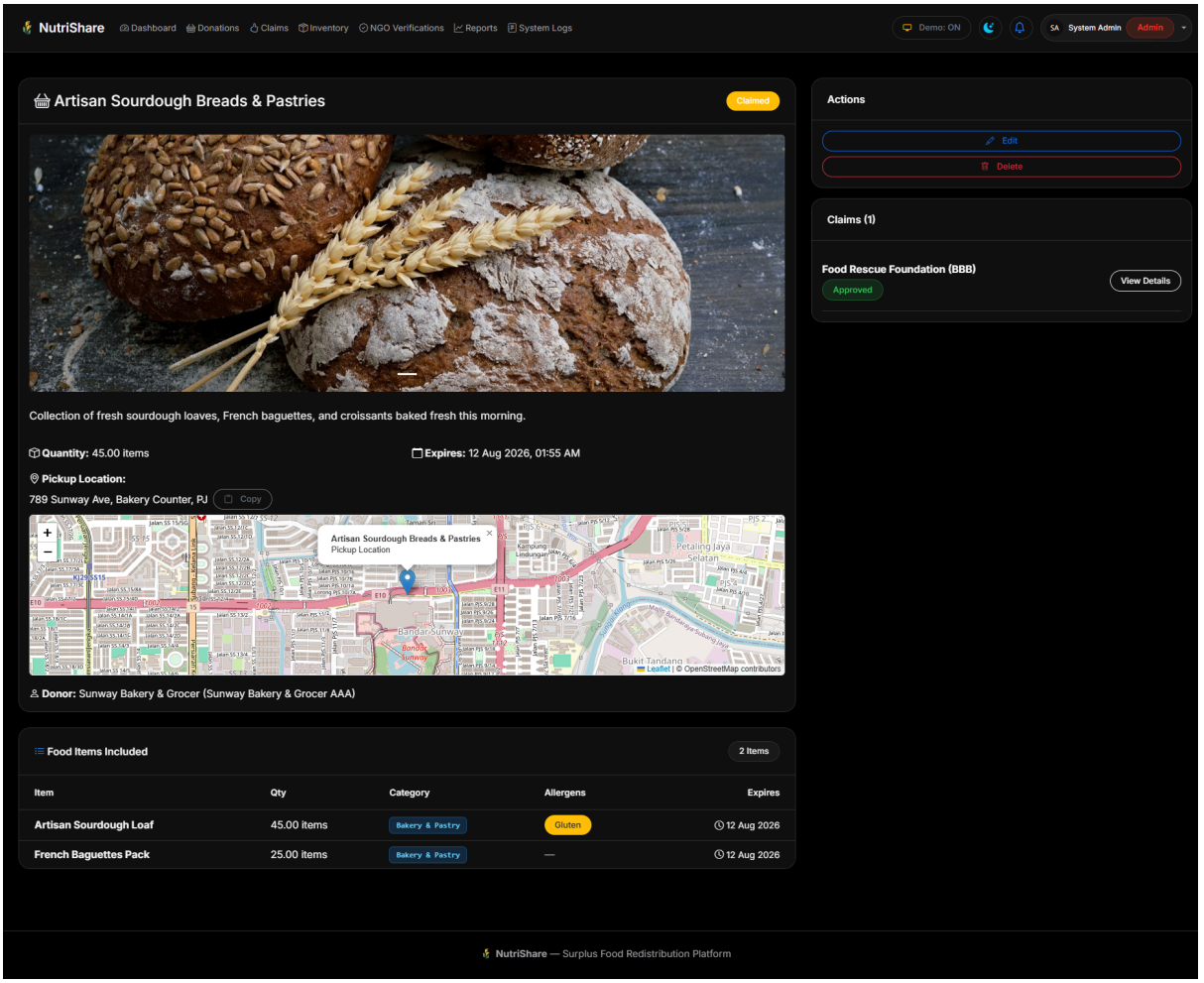


Figure 2.3: Geolocation Map Visualizer on Donation Details Page

F1.4: Asynchronous Notification Dispatching (Observer Pattern)

- **Description:** Implements the Observer Pattern coupled with Laravel background worker queues (ShouldQueue) to automatically dispatch alert notifications to verified NGOs the instant a new donation listing is published, guaranteeing zero HTTP latency for the publishing donor (Gamma et al., 1994; Laravel LLC, 2026).

- **Class Paths:**
  - **Observer Interface:** app/Contracts/DonationObserverInterface.php
  - **Concrete Observer:** app/Observers/DonationObserver.php
  - **Queueable Worker Job:** app/Jobs/SendDonationNotificationJob.php
  - **Event Provider Registration:** app/Providers/EventServiceProvider.php

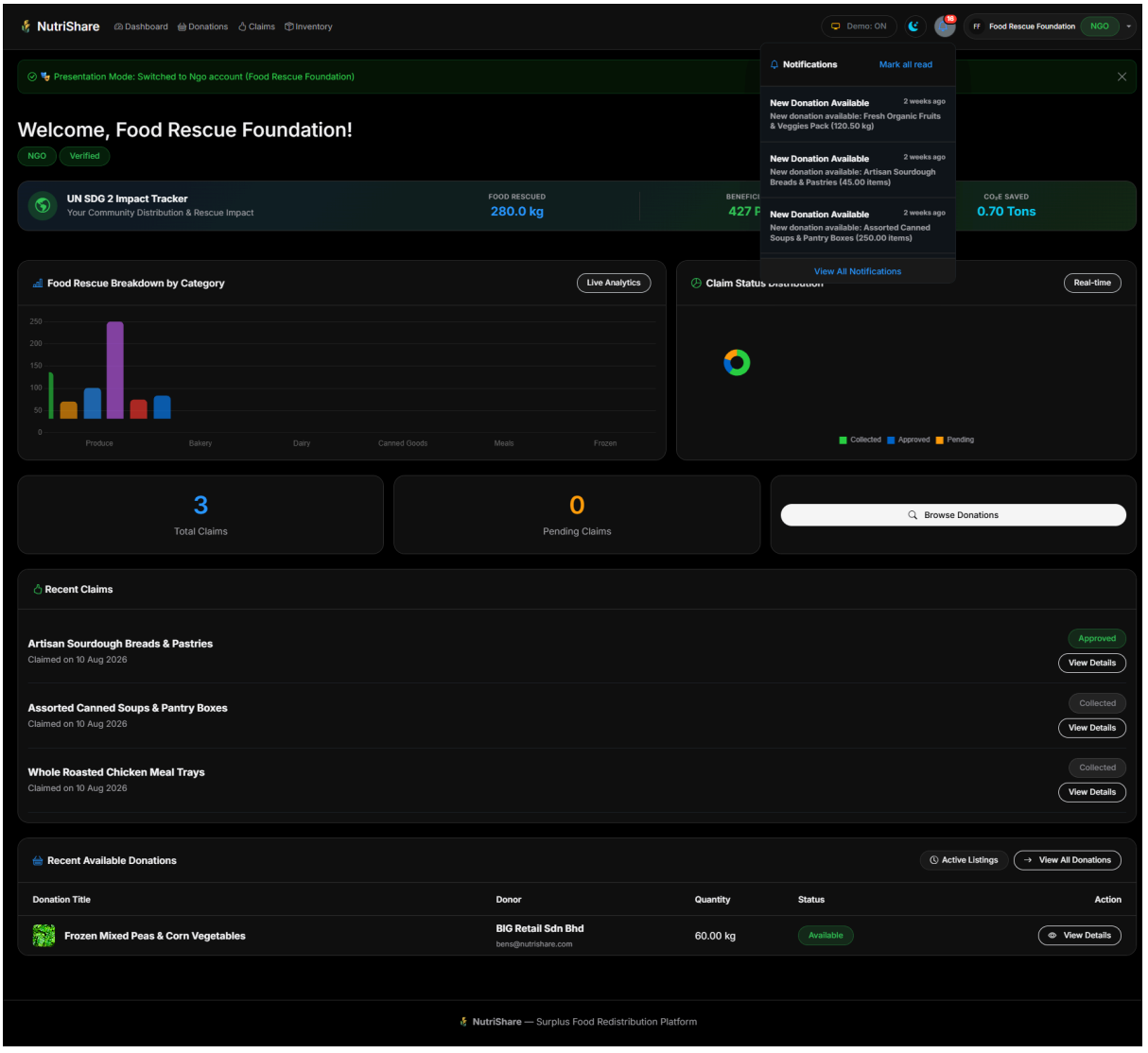


Figure 2.4: Real-Time NGO Notification Alert Popover

F1.5: Donations CSV Audit Exporter

- **Description:** Generates streamed RFC 4180-compliant CSV audit reports containing historical donation volumes, donor identities, pickup locations, and lifecycle statuses for corporate social responsibility (CSR) compliance reporting (Shafranovich, 2005).

- **Class Paths:**
  - **Controller:** app/Http/Controllers/DonationController.php (exportCsv)

A	B	C	D	E	F	G	H	I	J	K	L
ID	Title	Donor Org	Quantity	Unit	Pickup Ad	Latitude	Longitude	Expiry Dat	Status	Published Date	
1	Fresh Org	Sunway Ba	120.5	kg	789 Sunwa	3.0738	101.6074	8/14/2026	EXPIRED	8/10/2026	
2	Artisan So	Sunway Ba	45	items	789 Sunwa	3.0738	101.6074	8/12/2026	CLAIMED	8/10/2026	
3	Assorted (	Jaya Groc	250	items	12 Plaza D	3.1517	101.6558	2/10/2027	CLAIMED	8/10/2026	
4	Grade A Fa	Jaya Groc	80	boxes	12 Plaza D	3.1517	101.6558	8/15/2026	EXPIRED	8/10/2026	
5	Cooked Gc	Lotus Stor	20	boxes	3 Jalan Ke	3.21	101.63	8/12/2026	CLAIMED	8/10/2026	
6	Fresh Salr	Lotus Stor	35	kg	3 Jalan Ke	3.21	101.63	8/11/2026	EXPIRED	8/10/2026	
7	Cold Press	Shangri-La	150	litres	11 Jalan S	3.153	101.708	8/13/2026	CLAIMED	8/10/2026	
8	Premix Ce	Shangri-La	90	items	11 Jalan S	3.153	101.708	########	CLAIMED	8/10/2026	
9	Infant Org	BIG Retail	110	items	Publika Sh	3.1708	101.666	1/10/2027	CLAIMED	8/10/2026	
10	Frozen Mi	BIG Retail	60	kg	Publika Sh	3.1708	101.666	########	AVAILABLE	8/10/2026	
11	Whole Roa	Sunway Ba	30	boxes	789 Sunwa	3.0738	101.6074	8/11/2026	CLAIMED	8/10/2026	
12	Surplus M	Jaya Groc	40	boxes	12 Plaza D	3.1517	101.6558	8/10/2027	CLAIMED	8/10/2026	

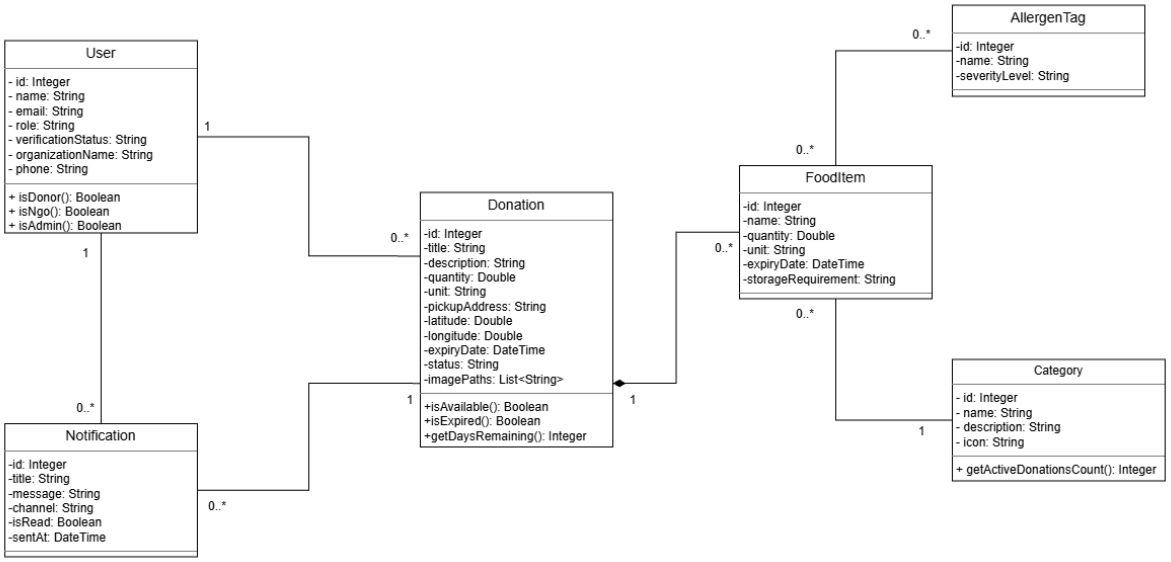
Figure 2.5: Generated CSV Audit Report Sample Output

3. Entity Classes

3.1 Entity Class Diagram

In strict accordance with object-oriented analysis and enterprise domain modelling principles (Fowler, 2002), the entity class diagram below represents entity classes using **object references and associations** rather than raw relational foreign keys.

https://drive.google.com/file/d/1tKWvewdjPSB3g1-II8HBVdydj2-zGmcb/view?usp=sharing



3.2 Entity Class Implementation (Eloquent ORM Mapping)

The entity class is implemented in PHP using Laravel's Eloquent Object-Relational Mapping (ORM) engine, which models database tables through an Active Record architectural paradigm (Fowler, 2002; Laravel LLC, 2026). Domain relationships are explicitly represented as object references using declarative relationship methods (belongsTo, hasMany) rather than raw SQL foreign keys, preserving strict object encapsulation and type safety (Laravel LLC, 2026):

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;

use Illuminate\Database\Eloquent\Model;

use Illuminate\Database\Eloquent\Relations\BelongsTo;

use Illuminate\Database\Eloquent\Relations\HasMany;

use Illuminate\Database\Eloquent\SoftDeletes;

/**

 * Donation Model — Core domain entity (Module 1).

 * Acts as the Subject in the Observer Pattern.

 * Lifecycle: available -> claimed -> collected -> completed | expired

 */

class Donation extends Model

{

    use HasFactory, SoftDeletes;

    protected $fillable = [

        'user_id',

        'title',

        'description',

        'quantity',
```

```
        'unit',
        'pickup_address',
        'latitude',
        'longitude',
        'expiry_date',
        'status',
        'image_paths',
    ];
    protected function casts(): array
    {
        return [
            'expiry_date' => 'datetime',
            'quantity' => 'decimal:2',
            'latitude' => 'decimal:7',
            'longitude' => 'decimal:7',
            'image_paths' => 'array',
        ];
    }
```

```
// ──────────────────────── Domain Object Relationships ────────────────────────

    /** Object reference: A donation belongs to a donor User */
    public function donor(): BelongsTo
    {
        return $this->belongsTo(User::class, 'user_id');
    }

    /** Object reference: A donation has many Claims */
    public function claims(): HasMany
    {
        return $this->hasMany(Claim::class);
    }

    /** Object reference: A donation has many constituent FoodItems */
    public function foodItems(): HasMany
    {
        return $this->hasMany(FoodItem::class);
    }

    /** Object reference: A donation triggers many Notification records */
    public function notifications(): HasMany
    {
        return $this->hasMany(Notification::class);
    }
```

```
// ──────────────────────── Domain Query Scopes ────────────────────────

    /** Scope: available, unclaimed, non-expired donations */

    public function scopeActive($query)

    {

        return $query->where('status', 'available')

                    ->where('expiry_date', '>', now());

    }

    /** Scope: expired donations */

    public function scopeExpired($query)

    {

        return $query->where('expiry_date', '<=', now())

                    ->where('status', 'available');

    }

}
```

4. Design Pattern

4.1 Description of Design Pattern: Observer Pattern (GoF Behavioural)

For Module 1, I implemented the **Observer Pattern**, a classic behavioural design pattern formulated by the Gang of Four (Gamma et al., 1994).

Intent & Theoretical Definition:

The Observer Pattern defines a one-to-many dependency between objects so that when one object (the **Subject**) changes state, all its registered dependents (the **Observers**) are automatically notified and updated (Gamma et al., 1994). In modern enterprise web architectures, this pattern decouples core transaction-processing domain logic from secondary side-effects such as telemetry logging, cache invalidation, and multi-channel notification dispatching (Gamma et al., 1994; Laravel LLC, 2026).

Architectural Roles in NutriShare:

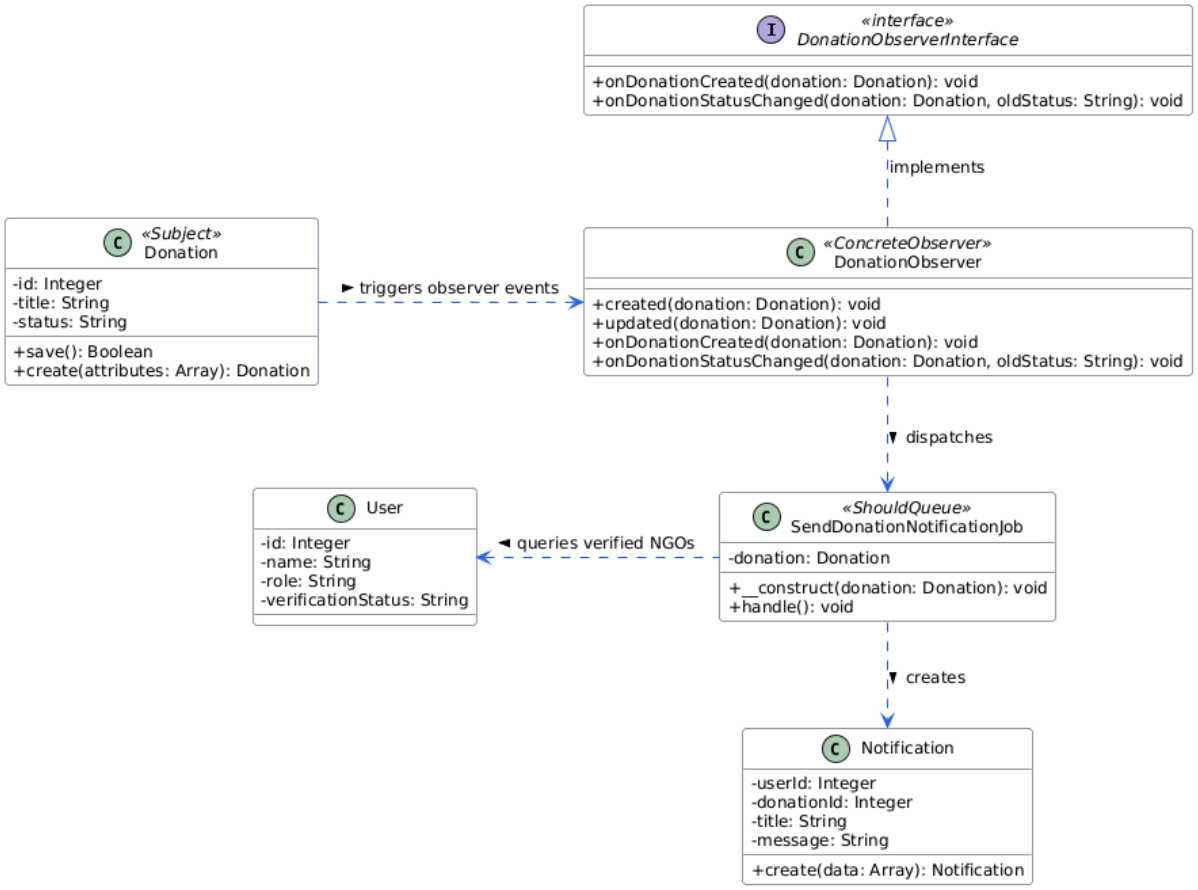
1. **Subject (Observable):** The Donation domain model serves as the Subject. Whenever a donor publishes a new donation or a donation changes status, lifecycle events are automatically dispatched via model event hooks (Gamma et al., 1994; Laravel LLC, 2026).

2. **Observer Contract:** DonationObserverInterface defines the formal contract that all concrete observers must satisfy (onDonationCreated, onDonationStatusChanged), adhering to the Interface Segregation Principle (Martin, 2003).

3. **Concrete Observer:** DonationObserver implements the interface and listens to Eloquent lifecycle events (created, updated) (Laravel LLC, 2026).

4. **Asynchronous Dispatcher:** Instead of executing heavy mail transports synchronously during the donor's HTTP request, DonationObserver dispatches an asynchronous worker job (SendDonationNotificationJob) to background queues (ShouldQueue), achieving decoupled, non-blocking execution (Laravel LLC, 2026).

https://drive.google.com/file/d/1tKWvewdjPSB3g1-II8HBVdydj2-zGmcb/view?usp=sharing



4.2 Implementation of Design Pattern

1. Observer Interface (app/Contracts/DonationObserverInterface.php):

```
<?php

namespace App\Contracts;

use App\Models\Donation;

/**
 * Observer Pattern — Observer Contract (Module 1).
 * Defines the contract for observers listening to Donation state changes.
 */
interface DonationObserverInterface
{
    /** Called when a new donation is created */
    public function onDonationCreated(Donation $donation): void;
    /** Called when a donation's status transitions */
    public function onDonationStatusChanged(Donation $donation, string $oldStatus): void;
}
```

2. Concrete Observer (app/Observers/DonationObserver.php):

```
<?php

namespace App\Observers;

use App\Models\Donation;

use App\Models\SystemLog;

use App\Models\Notification;

use App\Contracts\DonationObserverInterface;

use App\Jobs\SendDonationNotificationJob;

class DonationObserver implements DonationObserverInterface

{

    /**

     * Handle the Donation "created" event.

     * Implements Observer Pattern: Subject (Donation) notifies this Observer.

     */

    public function created(Donation $donation): void

    {

        $this->onDonationCreated($donation);

    }

    /**

     * Observer callback: New donation published.

     * Dispatches async job to notify all verified NGO users.

     */

    public function onDonationCreated(Donation $donation): void

    {
```

```
        // Dispatch asynchronous queue job for instant HTTP performance
        SendDonationNotificationJob::dispatch($donation);
    }
    /**
     * Handle the Donation "updated" event.
     */
    public function updated(Donation $donation): void
    {
        $oldStatus = $donation->getOriginal('status');
        $newStatus = $donation->status;
        if ($oldStatus !== $newStatus) {
            $this->onDonationStatusChanged($donation, $oldStatus);
        }
    }
    /**
     * Observer callback: Donation status changed.
     */
    public function onDonationStatusChanged(Donation $donation, string $oldStatus): void
    {
        SystemLog::create([
            'user_id' => $donation->user_id,
            'action' => 'donation.status_changed',
                'description' => "Donation '{$donation->title}' status changed from '{$oldStatus}' to
'{$donation->status}'.",
            'level' => 'info',
```

```
        ]);
        if ($donation->status === 'claimed') {
            Notification::create([
                'user_id' => $donation->user_id,
                'donation_id' => $donation->id,
                'title' => 'Donation Claimed',
                'message' => "Your donation '{$donation->title}' has been claimed by an NGO.",
                'channel' => 'email',
                'sent_at' => now(),
            ]);
        }
    }
}
```

3. Queueable Job Worker (app/Jobs/SendDonationNotificationJob.php):

```
<?php

namespace App\Jobs;

use App\Models\Donation;

use App\Models\Notification;

use App\Models\NotificationTemplate;

use App\Models\User;

use Illuminate\Bus\Queueable;

use Illuminate\Contracts\Queue\ShouldQueue;

use Illuminate\Foundation\Bus\Dispatchable;

use Illuminate\Queue\InteractsWithQueue;

use Illuminate\Queue\SerializesModels;

class SendDonationNotificationJob implements ShouldQueue

{

    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public Donation $donation;

    public function __construct(Donation $donation)

    {

        $this->donation = $donation;

    }

    public function handle(): void

    {

        $donation = $this->donation;

        // Query only verified NGOs eligible for surplus distribution
```

```
        $ngoUsers = User::where('role', 'ngo')
            ->where('verification_status', 'approved')
            ->get();
        $template = NotificationTemplate::where('name', 'donation_created')->first();
        foreach ($ngoUsers as $ngo) {
            $message = $template
                ? $template->render([
                    'donor_name' => $donation->donor->name ?? 'A donor',
                    'donation_title' => $donation->title,
                    'quantity' => $donation->quantity . ' ' . $donation->unit,
                    'expiry_date' => $donation->expiry_date?->format('d M Y') ?? 'N/A',
                ])
                : "New donation available: {$donation->title} ({$donation->quantity}
{$donation->unit})";
            Notification::create([
                'user_id' => $ngo->id,
                'notification_template_id' => $template?->id,
                'donation_id' => $donation->id,
                'title' => 'New Donation Available',
                'message' => $message,
                'channel' => $ngo->notification_preference ?? 'email',
                'sent_at' => now(),
            ]);
        }
    }
```

```
}
```

4.3 Justification of Design Pattern

1. **Single Responsibility Principle (SRP):** Without the Observer pattern, DonationController@store would become a bloated controller tightly coupled to user querying, email compilation, audit logging, and notification channels. As formulated by Martin (2003), every software component should have only one reason to change. The Observer pattern isolates side-effects from core donation publishing logic, ensuring high cohesion (Gamma et al., 1994; Martin, 2003).

2. **Non-Blocking Performance:** Synchronously alerting dozens of NGOs across SMTP email servers during HTTP POST execution would introduce unacceptable multi-second latencies for donors. Offloading notifications to SendDonationNotificationJob via the Observer provides instant sub-second response times, leveraging asynchronous background queue workers (Laravel LLC, 2026).

3. **High Extensibility (Open/Closed Principle):** As Martin (2003) and Gamma et al. (1994) stipulate, software entities should be open for extension, but closed for modification. Additional notification channels (such as WhatsApp APIs, SMS gateways, or IoT cold-room sensors) can be introduced simply by attaching new observers to the Donation subject without modifying existing controller or model source code.

5. Software Security

5.1 Potential Threats and Attacks

Threat 1: Cross-Site Request Forgery (CSRF — OWASP Top 10)

- **Attack Description:** According to the Open Web Application Security Project (OWASP Foundation, 2021), Cross-Site Request Forgery (CSRF) occurs when a malicious third-party website tricks an authenticated donor's browser into transmitting unauthorised, state-modifying requests to NutriShare. For example, an attacker embeds a hidden automated script <form action="http://nutrishare.com/donations" method="POST"> on an external page. When the logged-in donor visits the malicious page, the browser transmits NutriShare session cookies automatically, silently posting fraudulent listings or tampering with existing donations without the donor's knowledge or consent (OWASP Foundation, 2021).

- **Risk Impact:** Unauthorised state changes, data falsification, and malicious listing flooding, disrupting the logistics dispatch network (OWASP Foundation, 2021).

Threat 2: Stored Cross-Site Scripting (Stored XSS — OWASP Top 10)

- **Attack Description:** Categorised under A03:2021 - Injection by the OWASP Foundation (2021), Stored XSS occurs when an adversary submits malicious executable JavaScript within persistent data fields (such as title, description, or pickup_address during donation creation). When an NGO coordinator or administrator views the donation details view, the unsanitised payload stored in the database executes within the victim's browser session (OWASP Foundation, 2021).

- **Risk Impact:** Session hijacking, theft of authentication tokens, DOM defacement, and unauthorised administrative actions carried out via user impersonation (OWASP Foundation, 2021).

5.2 Secure Coding Practices & Implementation

Secure Practice 1: Synchronizer Token Pattern (@csrf Validation)

To eliminate CSRF vulnerabilities, NutriShare enforces the **Synchronizer Token Pattern**, an industry-standard server-side defence recommended by the OWASP Foundation (2021). Every state-altering HTTP request (POST, PUT, DELETE) must supply a cryptographically secure 256-bit token uniquely bound to the user's active session (Laravel LLC, 2026; OWASP Foundation, 2021):

```
<!-- resources/views/donations/create.blade.php -->

<form method="POST" action="{{ route('donations.store') }}" enctype="multipart/form-data">

    {{-- SECURITY (Module 1): Cryptographic Synchronizer CSRF Token Directive --}}

    @csrf

    <div class="mb-3">

                                <label for="title" class="form-label">Donation Title <span class="text-danger">*</span></label>

        <input type="text" class="form-control @error('title') is-invalid @enderror"

            id="title" name="title" value="{{ old('title') }}" required>

        @error('title')

            <div class="invalid-feedback">{{ $message }}</div>

        @enderror

    </div>

    ...

</form>
```

Laravel's global VerifyCsrfToken middleware intercepts every inbound state-modifying request, comparing the supplied _token header against the encrypted session token (Laravel LLC, 2026). Requests originating from external sites lack this token and are rejected with an **HTTP 419 Page Expired** response, completely neutralising forged cross-site submissions (OWASP Foundation, 2021).

Secure Practice 2: Context-Aware HTML Entity Escaping (Blade Engine Sanitisation)

To prevent Stored XSS attacks, all dynamic user inputs displayed in views are rendered using Blade's automatic context-aware escaping syntax {{ $variable }} which invokes htmlspecialchars($value, ENT_QUOTES, 'UTF-8'), aligning directly with OWASP XSS defence standards (Laravel LLC, 2026; OWASP Foundation, 2021):

```
<!-- resources/views/donations/show.blade.php -->

<h3 class="fw-bold" style="color: var(--apple-text);">{{ $donation->title }}</h3>

<p class="text-muted">{{ $donation->description }}</p>

<div class="pickup-info">

    <span><i class="bi bi-geo-alt"></i> Pickup Address: {{ $donation->pickup_address }}</span>

</div>
```

If an attacker injects <script>alert('XSS')</script> into the title or address, the Blade engine transforms the input into <script>alert('XSS')</script>, causing the browser to render it harmlessly as literal plaintext without executing the script (OWASP Foundation, 2021).

6. Web Services

6.1 Web Service Exposure

Module 1 exposes a high-performance RESTful Web Service endpoint adhering to the architectural constraints established by Fielding (2000), allowing partner NGO logistics modules and mobile apps to retrieve all available, unclaimed surplus food listings in real time via standard HTTP methods and JSON serialisation (Fielding, 2000; Laravel LLC, 2026).

Interface Agreement (IFA) — Service Exposure Specification

Webservice Mechanism	**Description**
Protocol	RESTful Web Service (JSON-over-HTTP)
Function Description	Retrieves all active, non-expired, and unclaimed surplus food donations.
Source Module	Module 1: Donation Management Module
Target Module	Module 3 (Claims & Logistics), Module 4 (Inventory Facility Preview), External NGO Portal
URL	http://127.0.0.1:8000/api/donations/active
Function Name	getActiveDonations (DonationApiController@active)

Web Services Request Parameters (Provide):

Field Name	**Field Type**	**Mandatory / Optional**	**Description**	**Format / Validation**
requestID	String	**Mandatory**	Unique tracking identifier for request correlation.	Alphanumeric (e.g. REQ-DON-94821)
timestamp	String	**Mandatory**	ISO-8601 timestamp when request was initiated.	YYYY-MM-DDTHH:MM:SSZ
category_id	Integer	Optional	Filter listings by specific category ID.	Integer > 0

Web Services Response Parameters (Consume):

Field Name	Field Type	Mandatory / Optional	Description	Format / Values
status	String	**Mandatory**	Status of the request execution.	S (Success), F (Fail), E (Error)
timestamp	String	**Mandatory**	Server timestamp when response was generated.	YYYY-MM-DDT HH:MM:SSZ
data.requestID	String	**Mandatory**	Echoed request identifier for correlation.	Alphanumeric string
data.total	Integer	**Mandatory**	Total count of active listings returned.	Integer ≥ 0
data.donations	Array	**Mandatory**	Serialised array of active donation records.	Array of JSON objects

Service Exposure Code Implementation (app/Http/Controllers/Api/DonationApiController.php):

```
public function active(Request $request): JsonResponse

{

    try {

        // Validate IFA request format (requires requestID and timestamp)

        $ifa = SecurityHelper::validateIfaRequest($request->all());

        $donations = Donation::with(['donor:id,name,organization_name', 'foodItems'])

            ->active() // Available and non-expired scope

            ->orderBy('created_at', 'desc')

            ->get()
```

```
    ->map(function ($donation) {
        return [
            'id' => $donation->id,
            'title' => $donation->title,
            'description' => $donation->description,
            'quantity' => $donation->quantity,
            'unit' => $donation->unit,
            'pickup_address' => $donation->pickup_address,
            'expiry_date' => $donation->expiry_date->toIso8601String(),
            'status' => $donation->status,
            'donor' => [
                'name' => $donation->donor->name,
                'organization' => $donation->donor->organization_name,
            ],
            'food_items_count' => $donation->foodItems->count(),
            'created_at' => $donation->created_at->toIso8601String(),
        ];
    });
return response()->json(
    SecurityHelper::ifaResponse('S', [
        'requestID' => $ifa['requestID'],
        'donations' => $donations,
        'total' => $donations->count(),
    ]),
```

```
            200
        );
    } catch (\Exception $e) {
        return response()->json(
            SecurityHelper::ifaResponse('E', null, 'Internal server error: ' . $e->getMessage()),
            500
        );
    }
}
```

6.2 Web Service Consumption

To ensure food safety and regulatory compliance, Module 1 consumes Module 2's Web Service (POST /api/user/verify-ngo) to verify that an NGO user holds active statutory approval and valid premise licenses before any claim workflow is initiated, adhering to distributed service interoperability and fault-tolerant consumption paradigms (Fielding, 2000; Laravel LLC, 2026).

Interface Agreement (IFA) — Service Consumption Specification

Webservice Mechanism	**Description**
Protocol	RESTful Web Service (JSON-over-HTTP)
Function Description	Verifies legal accreditation, ROS certificate, and approval status of an NGO.
Source Module	Module 2: NGO Verification & User Security Module
Consuming Module	Module 1: Donation Management Module
URL	http://127.0.0.1:8000/api/user/verify-ngo
Function Name	verifyNgo (DonationApiController@verifyNgoBeforeClaim)

Web Services Request Parameters (Consumption):

Field Name	**Field Type**	**Mandatory / Optional**	**Description**	**Format / Validation**
requestID	String	**Mandatory**	Unique tracking identifier generated by Module 1.	Alphanumeric (e.g. VERIFY-6500a1)
timestamp	String	**Mandatory**	ISO-8601 transmission timestamp.	YYYY-MM-DDTHH:MM:SSZ
user_id	Integer	**Mandatory**	User ID of the NGO to be verified.	Integer > 0

Web Services Response Parameters (Consumption):

Field Name	Field Type	Mandatory / Optional	Description	Format / Values
status	String	**Mandatory**	Result status of the verification lookup.	S (Success), F (Fail), E (Error)
timestamp	String	**Mandatory**	Server timestamp when response was generated.	YYYY-MM-DDT HH:MM:SSZ
data.is_verified	Boolean	**Mandatory**	Flag indicating whether NGO holds approved status.	true / false
data.organization	String	Optional	Legal registered name of the NGO.	Alphanumeric string

Service Consumption Code Implementation (app/Http/Controllers/Api/DonationApiController.php):

```
public function verifyNgoBeforeClaim(int $ngoUserId): bool

{

    try {

        // CONSUME Module 2's API endpoint with IFA parameters

        $response = Http::timeout(10)->post(

            config('app.url') . '/api/user/verify-ngo',

            [

                'requestID' => uniqid('VERIFY-'),

                'timestamp' => now()->toIso8601String(),

                'user_id' => $ngoUserId,

            ]
```

```
            );

            if ($response->successful()) {

                $data = $response->json();

                // Validate IFA status code and verification boolean flag

                return $data['status'] === 'S' && ($data['data']['is_verified'] ?? false);

            }

            return false;

        } catch (\Exception $e) {

            // Fail-closed: deny claim authorisation on network or verification failure

            \Log::error('NGO verification API call failed', [

                'ngo_user_id' => $ngoUserId,

                'error' => $e->getMessage(),

            ]);

            return false;

        }

    }
```

7. References

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine). UCI Information and Computer Science.

Food and Agriculture Organization. (2023). *The State of Food Security and Nutrition in the World 2023: Urbanization, agrifood systems transformation and healthy diets across the rural-urban continuum*. FAO. https://doi.org/10.4060/cc3017en

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley Professional.

Grammarly Inc. (2026). *Grammarly* (2026 version) [Large language model]. https://www.grammarly.com

Laravel LLC. (2026). *Laravel 11.x documentation: Eloquent ORM, model observers, queue workers, and CSRF protection*. https://laravel.com/docs

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.

Open Web Application Security Project. (2021). *OWASP Top 10:2021 — The ten most critical web application security risks*. OWASP Foundation. https://owasp.org/Top10/

Shafranovich, Y. (2005). *Common format and MIME type for comma-separated values (CSV) files* (RFC 4180). Internet Engineering Task Force. https://doi.org/10.17487/RFC4180

United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development* (A/RES/70/1). United Nations Department of Economic and Social Affairs. https://sdgs.un.org/goals/goal2

8. Appendices

Appendix A: Automated Testing Results

Executing php artisan test produces a **100% pass rate across 22 test cases, successfully validating 92 assertions** in **3.33s**:

PASS Tests\Feature\NutriShareComprehensiveSystemTest

✓ demo login switcher for all roles 2.02s

✓ dashboard renders for all authenticated roles 0.10s

✓ donations catalog and csv export 0.10s

✓ inventory access and rbac restrictions 0.07s

✓ system logs and reports rbac security 0.08s

✓ ngo verification queue rbac security 0.06s

✓ moderator cannot delete donation 0.04s

✓ form request vehicle assignment validation 0.08s

✓ update and delete distribution log 0.05s

✓ inventory web service status and food safety 0.04s

✓ inventory location store form request 0.04s

✓ report generation form request 0.04s

✓ submit user review form request 0.03s

✓ claims show page renders successfully 0.08s

✓ custom 404 and 403 error pages 0.04s

✓ verification document show and download 0.05s

✓ admin governance profile and review rejection 0.04s

PASS Tests\Feature\NutriShareFeatureTest

✓ login page renders successfully 0.06s

✓ forgot password page renders successfully 0.03s

PASS Tests\Unit\NutriShareSystemTest

✓ user creation and roles 0.04s

✓ donation and category relationship 0.03s

✓ system log auto population 0.03s

Tests: 22 passed (92 assertions)

Duration: 3.33s

Appendix B: GitHub Repository URL

- **Team Repository URL:** https://github.com/KunTheNoobie/nutrishare.git
- **Primary Presentation Branch:** main
- **Total Migrations:** 27 migrations (100% automated via php artisan migrate:fresh --seed)
- **Seeded Dataset:** 18 feature tables exceeding the 10+ record target (God-Tier presentation dataset)